Relational Data Records

CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

Learning Objectives

- Explain database normalization and why data is split into related models
- Add a `ForeignKey` field to a Django model to link two database tables
- Build an HTML form using Django's `ModelForm` class in `forms.py`
- Handle form submission with `CreateView`; override `form_valid()` to set the FK
- Store uploaded images with `models.ImageField` and configure `MEDIA_ROOT`/`MEDIA_URL`
- Use `get_success_url()` or `reverse()` to redirect after a successful form submission

Relational Data Models

A model (i.e., database table) describes a collection of data attributes (i.e., database fields). Example: `Article` model (table).

If we created one giant model in our application, we would have:

- Fields that are not applicable or irrelevant for some records
- Fields repeated with the same values across records

A database is normalized to create data models for each type of record, with relationships/interactions among tables.

Relationships Among Models (tables)

Records of one model (table) can be related to records of other models (tables) in several ways:

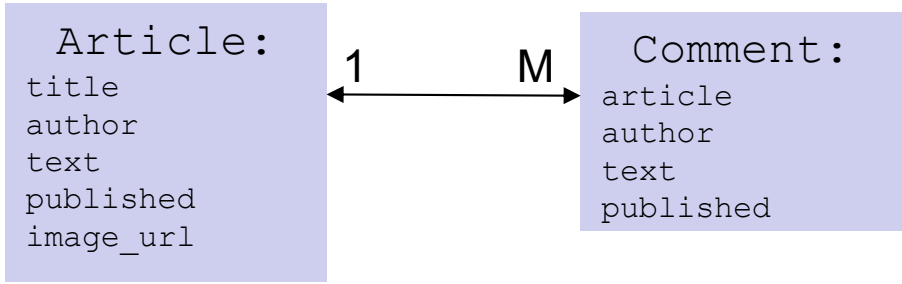
- One-to-one: a record of one model corresponds to exactly one record from another model.
- One-to-many: a record of one model relates to many records of another model.
- Many-to-many: many records of one model correspond to many records of another model.
  - In practice, this is implemented by creating another model to describe the relationship as two different one-to-many relationships.

Refining our Data Model

Old `models.py`



New `models.py`



An `Article` can have 0 … many `Comment`s

We model this relationship as a ForeignKey relationship:

- the `Article` is a foreign key to a `Comment`

BOSTON UNIVERSITY

Primary Key/Foreign Key

The primary key uniquely identifies one instance of a model (i.e., one database record).

- The Django ORM automatically creates a primary key for all models using the attribute `id` and the pseudonym `pk`.

The foreign key is an attribute of a model (i.e., a database field) that defines the relationship between one model and another.

- The Django ORM automatically creates the foreign key, based on our specifying an attribute of the type `models.ForeignKey`.

Defining a ForeignKey in models.py

`models.ForeignKey(OtherModel, on_delete=models.CASCADE)` links two models

`on_delete=CASCADE` — when the parent record is deleted, related records are also deleted

Django auto-creates the database column (e.g. `article_id`) and a reverse relation

```
class Article(models.Model):
    title     = models.TextField(blank=False)
    author    = models.TextField(blank=False)
    text      = models.TextField(blank=False)
    published = models.DateTimeField(auto_now=True)

class Comment(models.Model):
    article   = models.ForeignKey(Article, on_delete=models.CASCADE)
    author    = models.TextField(blank=False)
    text      = models.TextField(blank=False)
    published = models.DateTimeField(auto_now=True)
```

Migrations

A migration is a set of instructions to modify the database.

- When we make changes to our Models, the Django **`makemigrations`** command will discover changes and create a migration file to keep track of them.

```
python manage.py makemigrations <appname>
```

- We can apply (or unapply) migrations to modify the database itself, using the **`migrate`** command.

```
python manage.py migrate
```

Forms and Model Data

CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

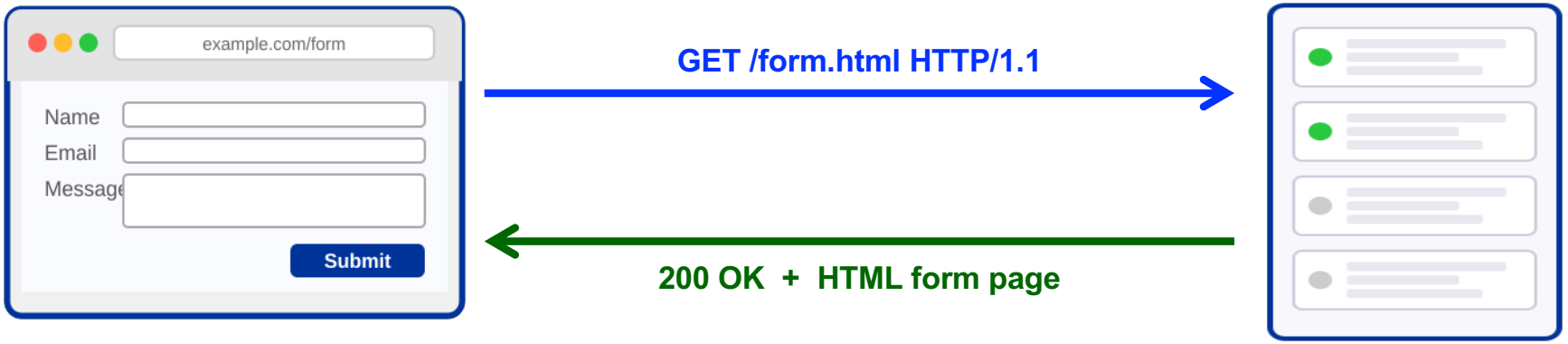
Generic Views

A generic view is a class which implements some common features, such as:

- `TemplateView`: Renders a template
- `ListView`: Showing a listing of all records of a model
- `DetailView`: Show a single instance of a model
- `CreateView`: Creates a new instance of a model
- `UpdateView`: Updates an instance of a model
- `DeleteView`: Deletes an instance of a model

STEP 1

HTTP GET — Requesting a Form



Client (Web Browser) Server

HTTP Request

```
GET /form.html HTTP/1.1
Host: example.com
Accept: text/html
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html

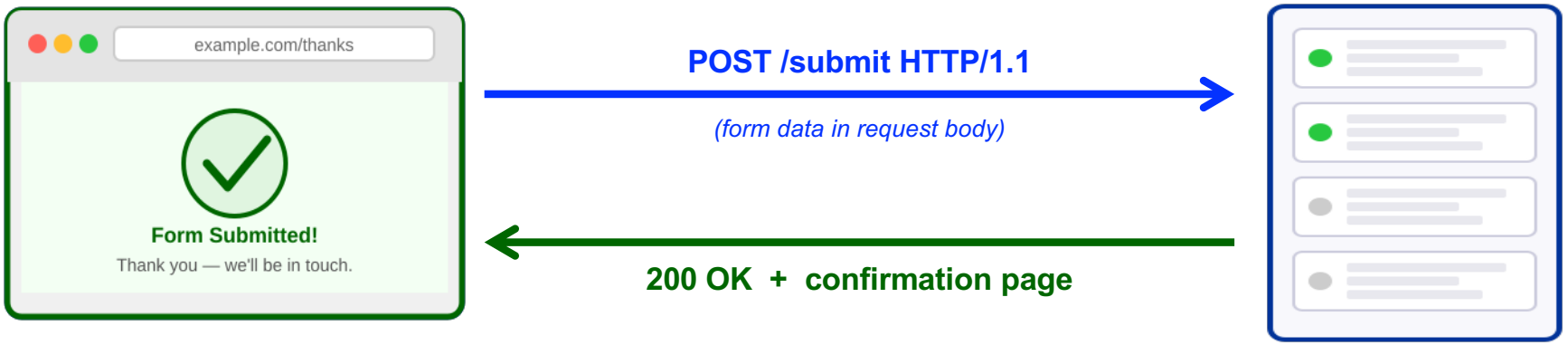
<form method="post"> … </form>
```

Note: GET requests have **no request body.** They only retrieve data from the server.

Key idea: The browser sends a GET request; the server responds with HTML containing a `<form method="post">` element. The browser renders the form — the same browser window that made the request.

STEP 2

HTTP POST — Submitting Form Data



Client (Web Browser) — Server

HTTP Request

```
POST /submit HTTP/1.1
Host: example.com
Content-Type: application/
  x-www-form-urlencoded

name=Alice+Smith
email=alice%40bu.edu
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Form submitted!</h1>
<p>Thank you, Alice.</p>
```

GET vs POST

GET: data in URL query string — visible, bookmarkable

POST: data in request body — hidden from URL, no size limit

Key idea: When the user clicks Submit, the browser sends a POST request. Form values travel in the **request body** — not the URL. The server processes them and responds; the same browser window displays the confirmation page.

STEP 1

Django CreateView — The 4 Components

Four files wire together to serve and process the HTML form:

urls.py

Maps the URL /create_comment to the view

```
path(
    'create_comment/',
    CreateCommentView.as_view(),
    name='create_comment'
)
```

views.py

Handles HTTP GET (show form) and POST (save data)

```
class CreateCommentView(CreateView):
    model         = Comment
    form_class    = CreateCommentForm
    template_name = 'create_comment.html'
    def get_success_url(self):
        return reverse('article_list')
```

forms.py

Generates an HTML form from the Comment model

```
class CreateCommentForm(ModelForm):
    class Meta:
        model   = Comment
        fields = ['author', 'text']
```

create_comment.html

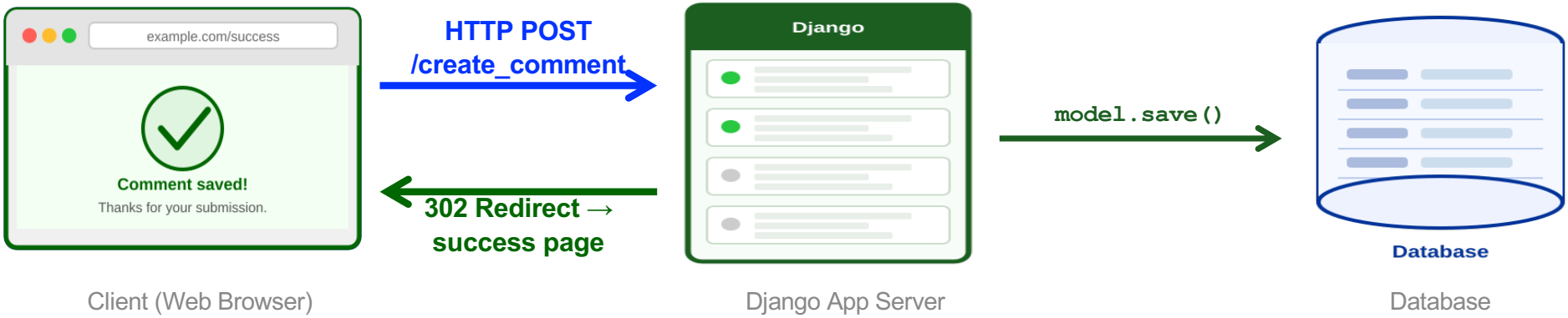
Renders the HTML form with a submit button

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Submit</button>
</form>
```

Key ideas: A `ModelForm` generates an HTML form directly from your model fields.
A `CreateView` handles both **HTTP GET** (display form) and **HTTP POST** (validate, save to DB, redirect) — almost no code required.

STEP 2

Django CreateView — HTTP POST + Database Save



HTTP Request (POST)

```
POST /create_comment HTTP/1.1
Host: example.com
Content-Type: application/
  x-www-form-urlencoded

author=Alice
text=Hello+world!
```

Django processes the POST:

1. URL routes request to CreateView
2. post() validates the form data
3. form_valid() calls form.save()
4. New object written to database
5. get_success_url() returns redirect target

What CreateView does for you:

- ✓ Validates all form fields
- ✓ Creates + saves the model object
- ✓ Redirects to `get_success_url()`
- ✓ If invalid: re-displays form with errors

`CreateView`: flow chart of method calls



Django calls automatically You typically override Entry point (always called)

CreateView: views.py

Subclass `CreateView`; set `model`, `form_class`, and `template_name`

Override `form_valid()` to attach the `ForeignKey` before the record is saved

Override `get_success_url()` to choose where to redirect after a successful save

```
# blog/views.py
from django.urls import reverse
from django.views.generic.edit import CreateView
from .models import Article, Comment
from .forms import CreateCommentForm

class CreateCommentView(CreateView):
    model         = Comment
    form_class    = CreateCommentForm
    template_name = 'blog/create_comment.html'

    def form_valid(self, form):
        article = Article.objects.get(pk=self.kwargs["pk"])
        form.instance.article = article
        return super().form_valid(form)

    def get_success_url(self):
        return reverse('article', kwargs={'pk': self.object.article.pk})
```

STEP 2

What page to show after form submission?

- `get_success_url` is a method on a view class. It generates a URL to redirect to after the form submission is successful.
- For example:

```
def get_success_url(self) -> str:
    '''Return the URL to redirect to after successfully submitting form.'''
    #return reverse('show_all')
    return reverse('article', kwargs={'pk': self.kwargs['pk']})
```

Suppose this method is called on an instance of the model with primary key of 5. The result of the method will be:

http://localhost:8000/blog/article/5

STEP 2

Setting the foreign key on a `Comment`

- We need to satisfy the foreign key constraint before we can save the newly created Comment object.
- One way: use a URL parameter (e.g., `pk`)
- For example:

```
class CreatePostView(CreateView):
    def form_valid(self, form):
        '''
        Handle the form submission. We need to set the foreign key by
        attaching the Profile to the StatusMessage object.
        We can find the profile PK in the URL (self.kwargs).
        '''

        # find the profile identified by the PK in the URL
        profile = Profile.objects.get(pk=self.kwargs['pk'])
        # attach this profile as a FK to the Post instance
        form.instance.profile = profile
        post = form.save() # save the Post to database
```

Review:
programmatically generating a URL

Django provides 2 very similar ways to generate a URL:

- Scriptlet `url` function

```
{% url 'article' article.pk %}
```

- Django `reverse` function

```
reverse('article', kwargs={'pk': article.pk}
```


BOSTON UNIVERSITY

Generating a URL using the `url` scriptlet function

Consider this HTML template code…

```
<article>
    <a href="{% url 'article' a.pk %}">
    <img src="{{a.image_url}}" alt="{{a.image_url}}">
    </a>
```

And these URLs:

```
# all of the URLs that are part of this app
urlpatterns = [
    path(r'', views.RandomArticleView.as_view(), name="random"),
    path(r'show_all', views.ShowAllView.as_view(), name="show_all"),
    path(r'article/<int:pk>', views.ArticleView.as_view(), name="article"), ## NEW

]
```

Generating a URL using the `reverse` function

Consider the **`get_success_url`** method:

```
def get_success_url(self) -> str:
    '''Return the URL to redirect to after successfully submitting form.'''
    #return reverse('show_all')
    return reverse('article', kwargs={'pk': self.kwargs['pk']})
```

And these URLs:

```
# all of the URLs that are part of this app
urlpatterns = [
    path(r'', views.RandomArticleView.as_view(), name="random"),
    path(r'show_all', views.ShowAllView.as_view(), name="show_all"),
    path(r'article/<int:pk>', views.ArticleView.as_view(), name="article"), ## NEW

]
```

ImageField

CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

`models.ImageField` and attributes

The **`ImageField`** data type encapsulates the idea of storing an image in the Django **`media`** directory, and rendering it in our application.

Attributes:

- **`name`**: the file name of the image
- **`path`**: the full path to the file in the file system
- **`url`**: the URL to reach this file on the WWW

Configuring ImageField in settings.py

Django stores uploaded files in `MEDIA_ROOT` and serves them at the `MEDIA_URL` path

Install `Pillow` first: `pip install Pillow` (required for `ImageField` to work)

Add a URL pattern so Django serves media files in development (`DEBUG=True`)

```
# cs412/settings.py
import os

MEDIA_ROOT = os.path.join(BASE_DIR, 'media/')
MEDIA_URL= "media/"  # note: no leading slash!

import socket
CS_DEPLOYMENT_HOSTNAME = 'cs-webapps.bu.edu'

if socket.gethostname() == CS_DEPLOYMENT_HOSTNAME:
    STATIC_URL = '/username/static/'
    MEDIA_URL = '/username/media/'
```

```
# cs412/urls.py
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... your URL patterns ...
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Example: using `models.ImageField`

models.py

cs412 > examples > django > blog > models.py > Article > image_file

```python
# blog/models.py
# define data models for the blog application
from django.db import models
from django.urls import reverse

# Create your models here.
class Article(models.Model):
    '''Encapsulate the data of a blog Article by an author.'''

    # define the data attributes of the Article object
    title = models.TextField(blank=True)
    author = models.TextField(blank=True)
    text = models.TextField(blank=True)
    published = models.DateTimeField(auto_now=True)
    # image_url = models.URLField(blank=True) # url as a string
    image_file = models.ImageField(blank=True) # an actual image
```

`mini_insta` case study: Data Models

CS412
Boston University

Aaron Stevens

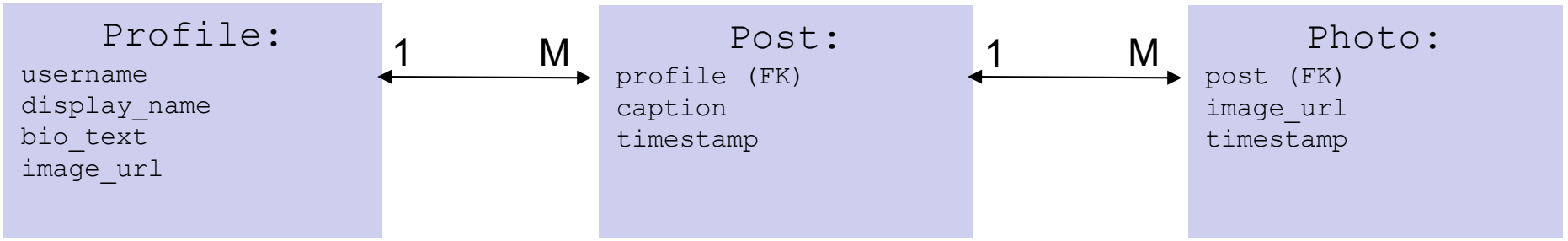
BOSTON UNIVERSITY

Refining our Data Model

Previous `models.py` (assignment 3)

Profile:
- username
- display_name
- bio_text
- image_url

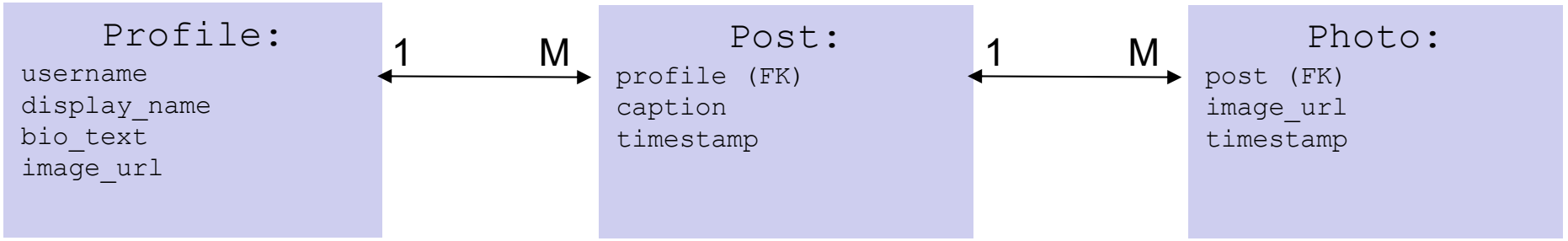
New `models.py` (assignment 4, task 1)



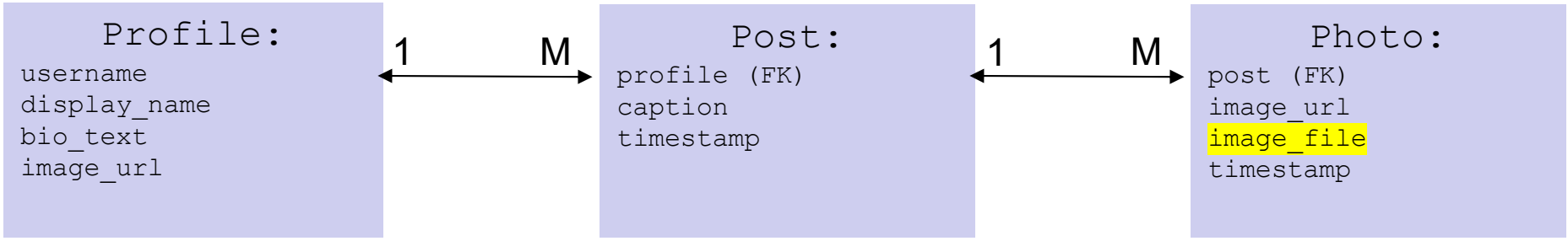
BOSTON UNIVERSITY

Refining our Data Model

Previous `models.py` (assignment 4, task 2)



Revised `models.py` (assignment 4, task 3)



Note: the `Photo.get_image_url` method should return the URL to the `image_file.url` (if exists) or else the `image_url`.

Summary

- Normalize data into separate models; use `ForeignKey` to define many-to-one relationships
- `models.ForeignKey(OtherModel, on_delete=models.CASCADE)` links models in `models.py`
- `ModelForm` auto-generates an HTML form from a model; define it in `forms.py`
- `CreateView` handles GET (show form) and POST (save record); override `form_valid()` to set FK
- `models.ImageField` stores uploaded files; set `MEDIA_ROOT` and `MEDIA_URL` in `settings.py`
- Redirect after form success using `get_success_url()` returning `reverse("view_name", ...)`